

Smart Expense Tracker Web Application

CP3407 Final Project Report

Prepared for James Cook University

Team capacity: 3 members x 4 working days = 12 person-days per iteration

Final version: August 2026

Final technical result: US1-US10 implemented; 40 automated tests passed; 15 system tests passed; 0 open critical/high-priority defects.

1. Executive Summary

The Smart Expense Tracker is a full-stack web application developed across three Agile iterations. The final solution provides secure registration and login, database-backed expense management, categorisation, budgets, alerts, reports, trend analysis, administrator account management, acceptance-feedback capture, automated testing, system testing, bug tracking and deployment support.

The earlier browser-only prototype has been upgraded to a Node.js server and executable relational SQLite database. Passwords are salted and hashed with bcrypt, sessions are verified by the server, and administrator actions are protected by role-based authorization. The committed user-story scope US1-US10 is implemented and tested.

2. Problem, Users and Requirements

The system responds to the needs identified in Practical 2: students with limited allowances, working adults managing salaries and bills, and small-business owners handling many transactions. The common problems are forgotten small expenses, weak categorisation, overspending and limited visibility of monthly trends.

US	Feature	Priority	Estimate	Iteration	Final status
US1	Register accounts	High	Final integration	Final	Done
US2	Secure login	High	Final integration	Final	Done
US3	Add expenses	High	6 person-days	1	Done
US4	Edit and delete expenses	High	6 person-days	1	Done
US5	Categorise/filter expenses	High	6 person-days	2	Done
US6	Set monthly budgets	High	6 person-days	2	Done
US7	Receive budget alerts	High	3 person-days	3	Done
US8	Monthly financial reports	Medium	3 person-days	3	Done
US9	Spending trend analysis	Medium	3 person-days	3	Done
US10	Administrator account management	Medium	3 person-days	3/final	Done

3. Agile Planning and Delivery

The team has three members and one team working day per week over four working days in each iteration. Capacity is therefore 12 person-days per iteration. User stories were ordered by dependency: expense data entry and maintenance first; categorisation and budgeting second; alerts, reports, trends and account administration third.

Iteration	Capacity	Delivered scope	Evidence
1	12 person-days	US3 and US4	Task board, burndown, code and tests
2	12 person-days	US5 and US6	Velocity update, TDD tests and burndown
3	12 person-days	US7-US10	Mock tests, system tests and bug tracking

- Todo, In Progress and Done states are documented for each iteration.

- Meaningful tasks, estimates, class/sequence designs, daily-commit guidance and pull-request review evidence are included.
- BUG-003 was fixed by sorting month keys before trend comparison and was protected by a regression test.

4. Architecture and Design

The final architecture separates the web interface, HTTP API, authentication/security functions, reporting/business logic and relational database. This improves SRP and DRY compared with the earlier single browser script. Editable Mermaid sources and online-editor links are included in the design evidence.

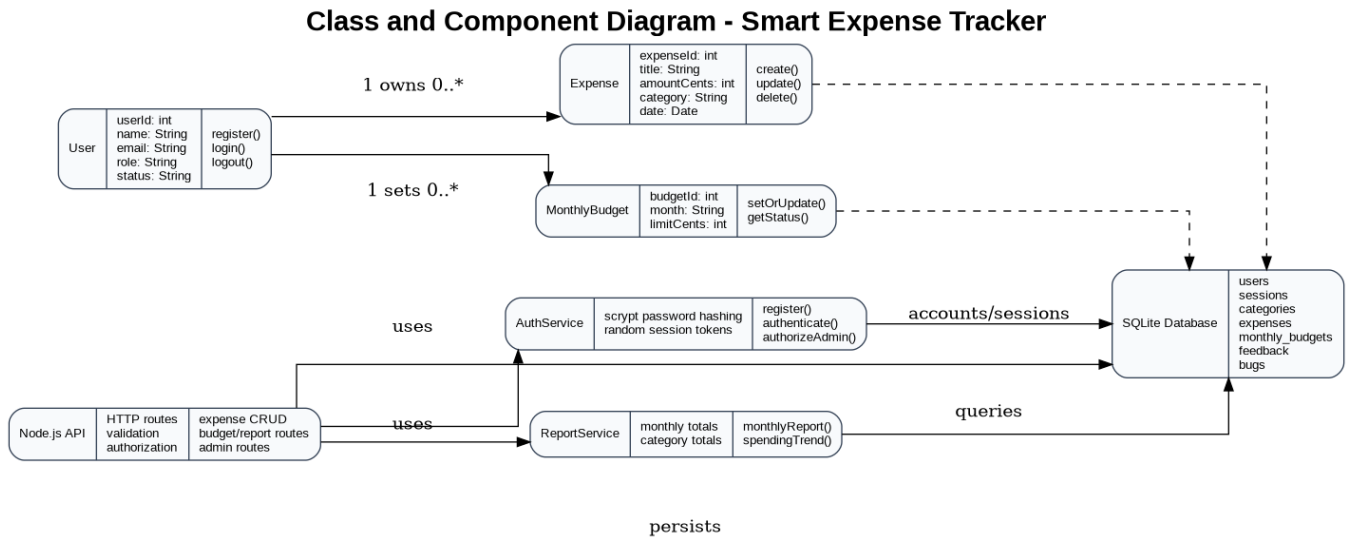


Figure 1. Final class and component design.

Relational Database ER Diagram

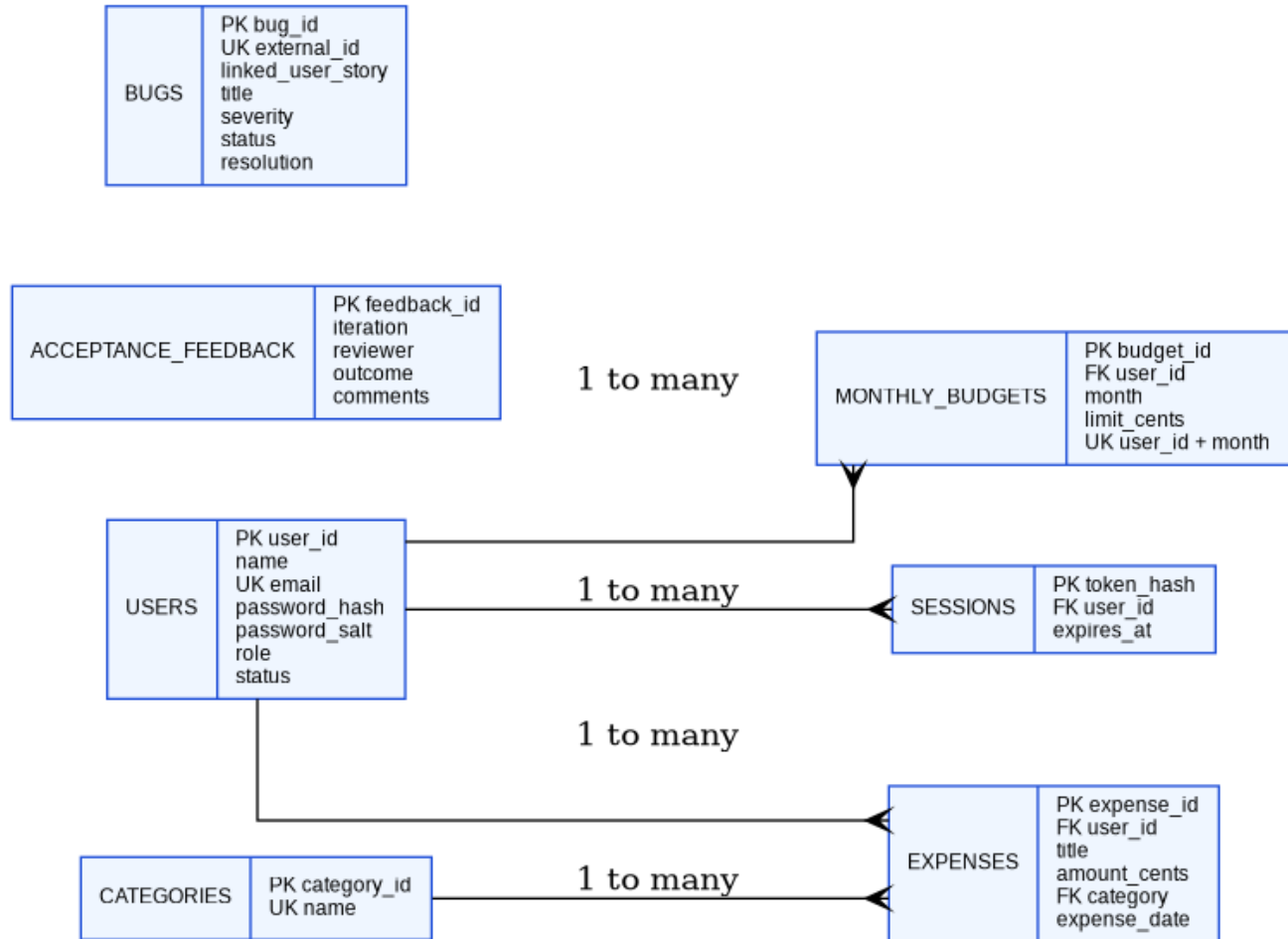


Figure 2. Final relational database entity-relationship design.

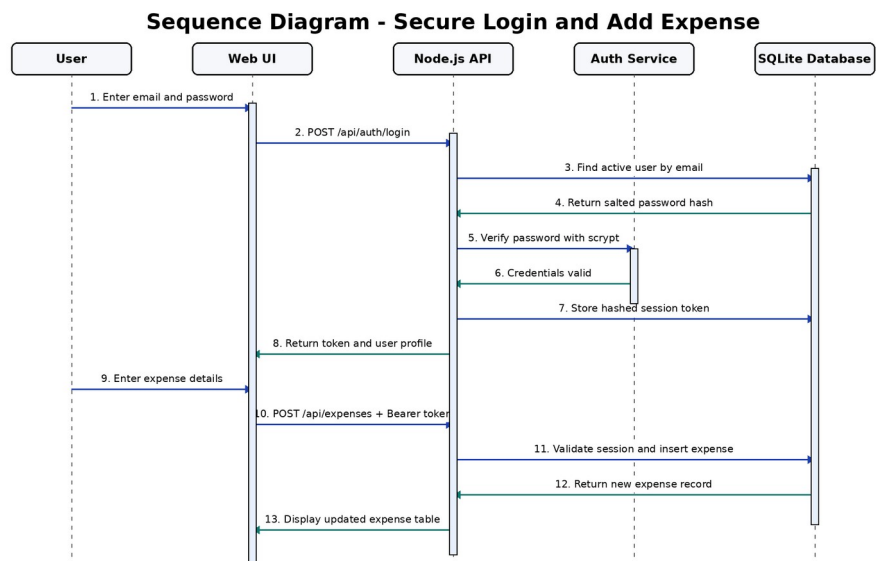


Figure 3. Secure login and expense-creation sequence.

5. Database Implementation

The application uses SQLite through Node.js. SQLite is a relational SQL database embedded in the server process. The runtime database is created automatically at data/expense-tracker.sqlite and uses foreign keys and cascading deletes.

Table	Purpose	Key controls
users	Account identity and authorization	Unique email; role/status checks; salted password hash
sessions	Authenticated sessions	Hashed token; expiry; user foreign key
categories	Controlled category names	Unique names
expenses	User expense records	User/category foreign keys; positive amount cents
monthly_budgets	Budget by user and month	Unique user+month; positive limit cents
acceptance_feedback	Real reviewer feedback	Iteration, reviewer, outcome, comments
bugs	Defect records	Linked US, severity, status and resolution

6. Security and Authentication

- Passwords are validated for minimum strength and stored only as salted script hashes.
- Login uses timing-safe comparison and returns a random session token; only a SHA-256 token hash is stored.
- Private routes validate an active session, while administrator routes require the admin role.
- Disabled users cannot log in; administrators can change role/status or delete accounts.
- HTTP responses apply Content-Security-Policy, X-Content-Type-Options, X-Frame-Options and Referrer-Policy headers.
- Production deployment must use HTTPS and an environment-provided administrator password.

7. Implementation and Demonstrable Features

Run the final application with `npm start` and open `http://127.0.0.1:3000`. The browser UI communicates only with authenticated API routes; application data is not persisted in localStorage.

Area	Implemented functions
Account	Register, login, logout, current session, disabled-account control
Expense	Add, edit, delete, validate, list and category filter
Budget	Set/update monthly budget and calculate safe/warning/over status
Reporting	Month-filtered total, category totals, top category and detailed rows
Trend	Chronologically sorted monthly totals and increase/decrease/stable result
Administration	List users, update role/status, disable and delete accounts
Feedback	Store genuine acceptance feedback in the relational database

8. Test-Driven Development and System Testing

The final test command is `npm test`. The suite contains 40 passing automated tests: the original 27 unit/mock tests, one BUG-003 regression test and 12 database/API integration tests. Tests cover all committed user stories and verify that passwords are not stored in plain text.

Test group	Count	Scope
Iteration 2 unit tests	15	US3-US7 expense, category, budget and

		alert logic
Iteration 3 unit/mock tests	12	US7-US10 reports, trends and mock login
BUG-003 regression	1	Unordered month data is sorted before trend analysis
Database/API integration	12	US1-US10 secure auth, persistence, CRUD, reports and admin

Final system-test summary: 15 Pass, 0 Conditional Pass, 0 Fail. Critical open bugs: 0. High-priority open bugs: 0.

9. Bug Tracking, Version Control and Tools

The repository contains a GitHub bug issue form, user-story task form, pull-request template and GitHub Actions CI workflow. A local Git repository with final-integration commits and tag v2.0-final is included. The repository owner must push this history and show the live Project, Issues, Pull Requests and Actions pages.

Tool	Use
HTML/CSS/JavaScript	Responsive web interface and client API calls
Node.js 22	HTTP server, crypto, automated tests and SQLite access
SQLite	Executable relational runtime database
Docker	Reproducible application container
GitHub Actions	Automated npm test on push and pull request
GitHub Issues/Projects/PRs	Task, defect, review and status tracking
Mermaid Live Editor	Editable online class, sequence and ER diagrams

10. Marking Rubric Review

Criterion	Evidence and status
Requirements	Correct US1-US10 scope, priorities, estimates and justified order - complete
Design	Architecture/class, sequence, ER and UI prototype sources - complete in package
Implementation/code	Database-backed full-stack application and all committed features - complete
Test	40 automated tests, 15 system tests, acceptance capture and closed bugs - complete
Version control	Git history, CI, issue forms and PR template - local evidence complete; push required
Building/tools	Node.js, SQLite, Docker, GitHub Actions and online diagrams - complete
Agile engineering	Three iterations, velocity, burndown, task boards, TDD and bug workflow - complete
Technical writing	Final report and project/GitHub pages documentation - complete

11. Week 10 Demonstration Plan

1. Run npm test and show 40 passing tests.
2. Run npm start and open the application.
3. Register and securely log in.
4. Demonstrate add/edit/delete/filter expenses.
5. Set budgets and show alert states.
6. Generate a monthly report and spending trend.
7. Log in as administrator and demonstrate role/status update, disable and delete.

8. Record acceptance feedback in the database.
9. Show database design, security, bug log and rubric review.
10. Show live GitHub/CI/Pages/deployment/collaborator evidence where completed.

12. External Evidence Requiring Real Accounts

The following cannot be truthfully completed inside a ZIP because they require access to the team repository, cloud account or a real reviewer. They must be completed and verified before submission:

- Push the repository and confirm the GitHub Actions run passes.
- Create or show the live GitHub Project with Todo, In Progress and Done; issues and pull requests.
- Publish the GitHub Pages documentation.
- Invite Dasheng LIU and confirm repository access.
- Deploy to a Node.js-capable host and record the public URL.
- Record genuine lecturer/client feedback during or after the demonstration.

13. Conclusion

The final package resolves the previously identified technical gaps: it uses a real relational database, secure server-side authentication, complete administrator account management, a fixed and regression-tested trend function, consistent user-story status documents, comprehensive automated/system testing and modern build/deployment tooling. The remaining items are external-account evidence and must not be fabricated.

References

- James Cook University. CP3407 Project assignment marking rubric. Updated 4 August 2026.
- James Cook University. CP3407 Projects requirements. Updated 17 May 2026.
- Node.js Documentation. Crypto and Test APIs.
- SQLite Documentation. About SQLite and appropriate uses.
- GitHub Documentation. Building and testing Node.js; issue forms; Projects; deployment to Azure App Service.
- Express Documentation. Production security best practices (used as a security checklist even though the final server uses Node.js built-in HTTP).
- Mermaid Live Editor and Mermaid open-source documentation.